

◇ DATA ◇ CODE ◇ MODELS ◇ INFRA

MLOps Best Practices

Shipping machine learning systems that stay reliable, reproducible, and maintainable — long after the demo.

Adapted from an article by Yadidiah Kanaparthi

Use ← → or Space to navigate_

QUIZ · MLFLOW

QUESTION 1 OF 4

What's the main difference between what Scikit-learn does and what an experiment-tracking tool like MLflow does?

- A Scikit-learn performs the actual training/prediction; MLflow records what happened during that process ✓
- B Scikit-learn logs metadata; MLflow trains the model
- C They do the same job, just with different syntax
- D MLflow replaces the need for a machine learning library entirely

Scikit-learn does the actual modeling work; MLflow just records what happened so you can find it again later.

QUIZ · MLFLOW

QUESTION 2 OF 4

In MLflow, which of these would you log as a Parameter rather than a Metric?

- A The learning rate used for training ✓
- B The model's test accuracy
- C A confusion matrix image
- D The trained model file

Parameters are inputs you fix before training starts — a learning rate is exactly that.

QUIZ · PREPROCESSING & ENSEMBLE METHODS

QUESTION 3 OF 4

When treating outliers with the IQR (interquartile range) method, "capping" (winsorizing) a value means:

A Deleting the row that contains it

C Replacing it with the column mean

B Replacing it with the nearest boundary of the acceptable range instead of removing the row ✓

D Ignoring it during training but keeping it for evaluation

Capping clips the value to the nearest acceptable boundary instead of throwing the row away.

QUIZ · PREPROCESSING & ENSEMBLE METHODS

QUESTION 4 OF 4

Conceptually, how does Boosting (e.g. XGBoost) differ from Bagging (e.g. Random Forest)?

- A Boosting trains trees independently in parallel; Bagging trains them sequentially
- B Boosting trains trees sequentially, each one correcting the errors of the previous ones; Bagging trains trees independently on bootstrap samples ✓
- C Boosting only supports regression tasks
- D There's no real difference between the two approaches

Boosting corrects errors sequentially, tree after tree; Bagging averages independent trees trained in parallel.

01 · FOUNDATIONS

What Is MLOps?

MLOps applies the discipline of DevOps — automation, testing, versioning, monitoring — to the unique challenges of machine learning: data that changes, models that decay, and experiments that must be reproducible.

FIG. 01

DevOps

Continuous integration, continuous delivery, infrastructure as code, automated testing.

FIG. 02

Data Engineering

Pipelines, quality checks, versioning, and governance for the data feeding every model.

FIG. 03

Machine Learning

Experimentation, training, evaluation, and the statistical nature of model behavior.

MLOps vs. DevOps: What's Different

MLOps grew out of DevOps — but machine learning introduces failure modes software engineering doesn't have.

Shared Principles

- Process automation
- Continuous integration & deployment
- Collaboration & communication
- Scalability & reliability
- Monitoring & feedback loops

Unique to MLOps

Reproducibility

Same code + same data can still give different results — version data, seeds, hyperparameters, and environment.

Scaling Training & Inference

Training needs specialized hardware (GPUs); serving must stay fast and cheap enough to be worth it.

Model Drift

Accuracy degrades as real-world data shifts away from what the model was trained on.

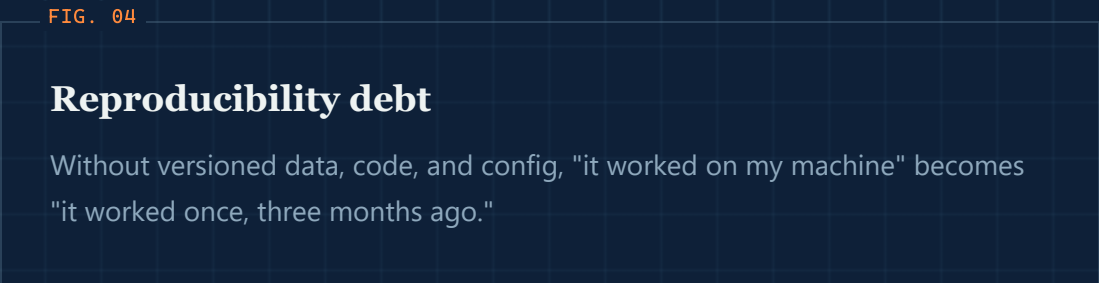
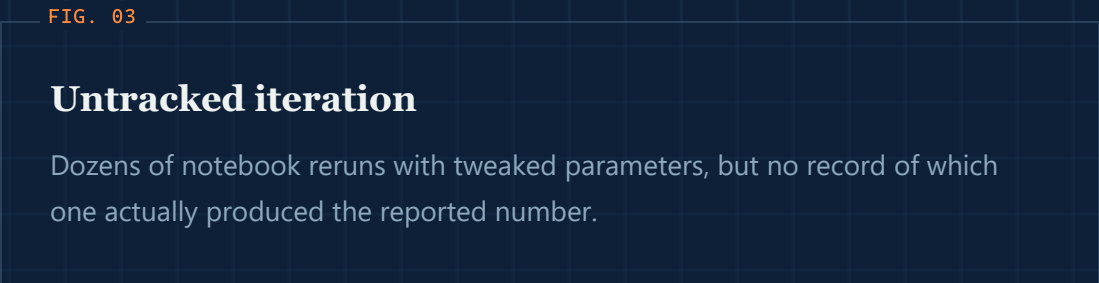
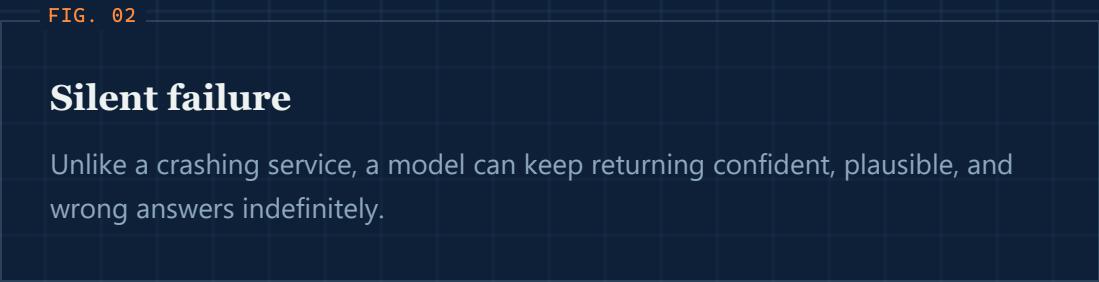
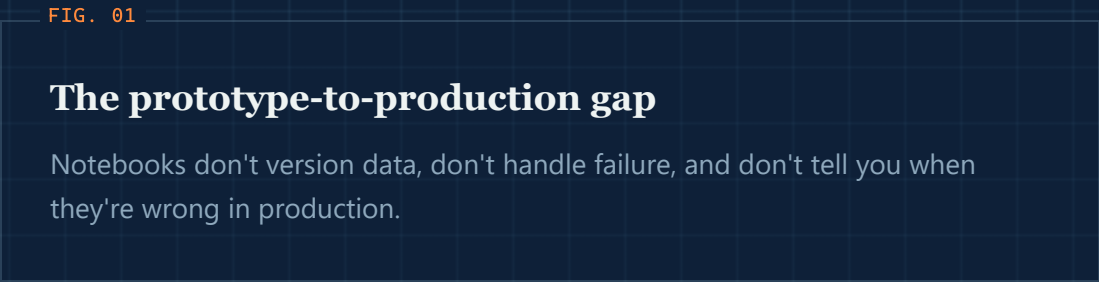
Team Composition

Researchers and production engineers have different skills and paces — they have to learn to work together.

01 · FOUNDATIONS

Why It Matters

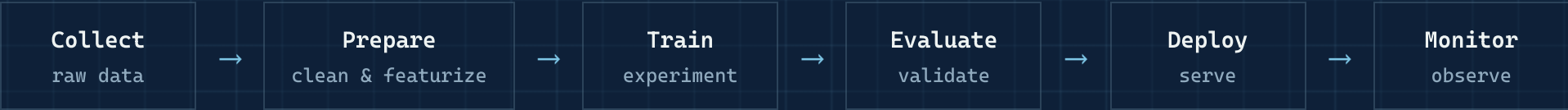
A notebook that produces a good metric is not a product. Most ML initiatives stall between prototype and production — not because the model is wrong, but because nothing around it is operationalized.



01 · FOUNDATIONS

The ML Lifecycle Is a Loop

Unlike traditional software, the lifecycle never really ends at "deploy." Production feedback continuously reshapes the next iteration.



↻ Retraining feeds back into Collect & Prepare

01 · FOUNDATIONS

The MLOps Tool Landscape

Beyond MLflow and DVC — a broader ecosystem covers every stage of the ML lifecycle.

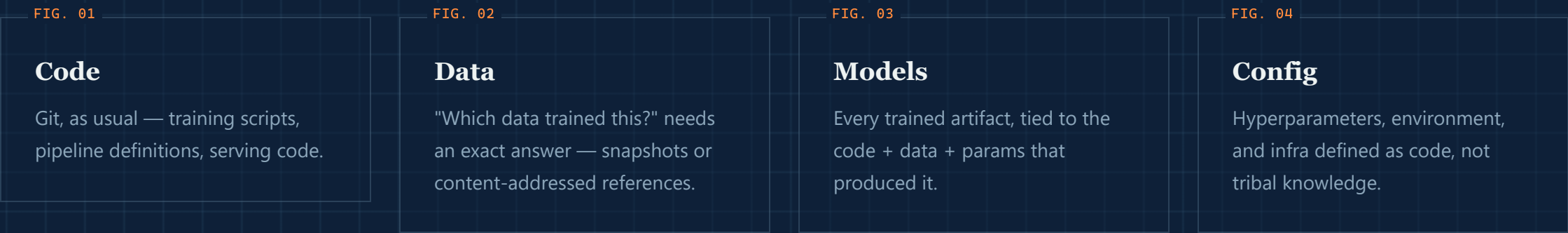


Source: InfluxData, "MLOps: A Comprehensive Guide to Machine Learning Operations"

02 · REPRODUCIBILITY

Version Everything

If you can't reconstruct exactly what produced a model, you can't debug it, audit it, or trust it.
Four things need a version history.



02 · REPRODUCIBILITY

Data Versioning & Lineage

Code review answers "what changed and why." Data needs the same answer — which snapshot, transformed how, by which job.

FIG. 01

Version like code

Tools such as DVC, LakeFS, or Delta Lake let you tag, diff, and roll back datasets the way Git handles source files.

FIG. 02

Track lineage

Record which raw sources, transformations, and jobs produced each downstream table or feature.

FIG. 03

Debug backwards

When a model misbehaves, lineage lets you walk back to the exact upstream change that caused it.

FIG. 04

Enable audits

Regulated industries need to prove exactly what data trained a production model, months later.

02 · REPRODUCIBILITY

DVC in Practice

Version datasets and pipelines alongside Git — reproducible by construction.

TERMINAL

```
# version a dataset
dvc add data/train.csv
git add data/train.csv.dvc
git commit -m "Add training data v1"

# run the full pipeline
dvc repro
```

DVC.YAML – PIPELINE DEFINITION

```
stages:
  preprocess:
    cmd: python src/data/preprocessing.py
    deps:
      - src/data/preprocessing.py
      - data/raw/transactions.csv
    outs:
      - data/processed/clean.csv
  train:
    cmd: python src/models/train.py
    deps:
      - src/models/train.py
      - data/processed/clean.csv
    metrics:
      - metrics.json
```

Every dataset version is tied to a Git commit – checkout any commit and dvc checkout restores the exact data that produced it.

03 · EXPERIMENTATION

Track Every Experiment

If a result isn't logged, it didn't happen. Experiment tracking turns ad-hoc notebook runs into a searchable, comparable history.

FIG. 01

Parameters

Hyperparameters, feature sets, random seeds — the recipe for each run.

FIG. 02

Metrics

Loss curves, accuracy, business metrics — logged per step, not just the final number.

FIG. 03

Artifacts

Model weights, plots, and evaluation reports attached to the run that produced them.

Tools in this space (e.g. MLflow, Weights & Biases) exist mainly to make "which run was that?" a solved problem.

03 · EXPERIMENTATION

MLflow in Action: Track Everything

A config-driven training function logs params, metrics, and the model itself on every run.

```
def train_model(config_path):  
    config = yaml.safe_load(open(config_path))  
    mlflow.set_experiment(config['experiment_name'])  
  
    with mlflow.start_run(run_name=config['run_name']):  
        mlflow.log_params(config['model_params'])  
  
        X_train, y_train = load_data(config['data_path'])  
        model = RandomForestClassifier(**config['model_params'])  
        model.fit(X_train, y_train)  
  
        mlflow.log_metric("accuracy",  
                          accuracy_score(y_val, model.predict(X_val)))  
        mlflow.sklearn.log_model(model, "model")
```

Compare experiments visually in the MLflow UI

Reproduce any run from its logged params + code version

Deploy models directly from the registry

03 · EXPERIMENTATION

Reproducible Environments

"Works on my machine" is not acceptable for a training job that needs to run identically today, next month, and on a teammate's laptop.

FIG. 01

Containers

Docker images pin the OS, drivers, and system libraries around the training/serving code.

FIG. 02

Locked dependencies

Exact, hashed package versions — not loose ranges — so a rebuild resolves identically.

FIG. 03

Infra as code

Clusters, GPUs, and networking defined in version-controlled templates, not clicked together by hand.

03 · EXPERIMENTATION

How to format your Ready-Production Project

Stop using flat notebooks, A production-ready project separates concerns — data, source code, configs, and tests each get their own home.

data/

raw, processed, and feature-engineered datasets

src/

data, features, models, evaluation code

configs/

hyperparameters & pipeline settings as code

tests/

automated validation for code and data

notebooks/

exploration only — never production logic

dvc.yaml

reproducible pipeline definition

```
ml-project/
├── data/
│   ├── raw/
│   ├── processed/
│   └── features/
├── notebooks/
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── evaluation/
├── configs/
├── tests/
├── mlruns/
├── dvc.yaml
└── requirements.txt
```

Testing an ML System

Unit tests alone don't catch a model that's statistically wrong. ML systems need their own test pyramid.

FIG. 01

Data tests

Schema, ranges, nulls, and distribution checks on every incoming batch.

FIG. 03

Behavioral tests

Invariance and directional-expectation checks — e.g. changing an unrelated field shouldn't flip the prediction.

FIG. 02

Unit tests

Feature transforms and pipeline code, tested like any other software.

FIG. 04

Integration tests

The full pipeline end-to-end, including serving latency and contract checks.

Best Practices

Five habits that separate teams that ship ML reliably from teams that don't.

- 1

Treat Config as Code
Never hardcode hyperparameters — use versioned YAML config files.
- 2

Automate Data Quality Checks
Validate every batch with tools like Great Expectations before training.
- 3

Implement Model Monitoring
Detect data drift automatically and trigger retraining (e.g. Evidently).
- 4

Version Everything
Code in Git, data in DVC, models in MLflow Registry, infra in Terraform.
- 5

Shadow Mode Deployments
Run candidate models alongside production, compare before promoting.

Common Pitfalls & How to Avoid Them

Four mistakes that quietly sink ML projects — and the fix for each.

FIG. 01

Training/Serving Skew

PROBLEM: Features computed differently in training vs. production.

SOLUTION: Use a feature store or shared feature-engineering code for both.

FIG. 03

Ignoring Model Decay

PROBLEM: Accuracy quietly degrades as the world changes.

SOLUTION: Schedule regular retraining and monitor performance continuously.

FIG. 02

Data Leakage in CV

PROBLEM: Shuffling before split lets future data leak into training.

SOLUTION: Use time-based splits for temporal data.

FIG. 04

Over-Engineering Too Early

PROBLEM: Building Kubernetes clusters before validating model value.

SOLUTION: Follow crawl → walk → run: start simple, add complexity as value is proven.

05 · PITFALLS

Fixing a Common Pitfall: Time-Based Splits

Random shuffling is the single most common source of data leakage in time-series problems.

WRONG

```
# Wrong - future data leaks into training
X_train, X_test = train_test_split(
    X, test_size=0.2, shuffle=True)
```

RIGHT

```
# Right - split strictly by time
split_date = df['date'].quantile(0.8)
train = df[df['date'] < split_date]
test = df[df['date'] ≥ split_date]
```

An MLOps Maturity Model

Maturity is a gradient, not a switch. Most organizations move through these stages one capability at a time.



Implementing MLOps: A Roadmap

A high-level path organizations follow to stand up an MLOps practice.

- 1

Establish current state & objectives
Baseline today's deployment time and model accuracy; set concrete improvement goals.
- 2

Build the MLOps team
Blend data science, engineering, ops, and domain expertise.
- 3

Define data governance
Set standards for collection, storage, versioning, quality, and compliance.
- 4

Select tools & platforms
Weigh build vs. buy, team experience, and community support.
- 5

Automate the deployment pipeline
Start with CI/CD for testing, tracking, and promoting models.
- 6

Iterate and improve
MLOps is ongoing — revisit objectives and raise the bar over time.

CLOSING

Key Takeaways

FIG. 01

Version everything

Code, data, models, and config — all reconstructible.

FIG. 02

Automate the pipeline

From data validation through deployment, not just training.

FIG. 03

Monitor in production

Drift and decay are the default; watch for them continuously.

FIG. 04

Share ownership

MLOps succeeds as a team practice, not a handoff.

Thank you.



Let's move on to today's challenge.